

5주차

# SELECT 기본

조회의 구조, WHERE 조건 검색, 정렬과 내장 함수

# 이번 주를 마치면

|      |                                            |
|------|--------------------------------------------|
| 목표 ① | SELECT 문의 기본 구조와 실행 순서를 설명한다               |
| 목표 ② | 별칭(AS)과 DISTINCT로 조회 결과를 가공한다              |
| 목표 ③ | 비교·논리 연산자, BETWEEN·IN·LIKE·IS NULL을 골라 쓴다  |
| 목표 ④ | NULL의 3값 논리(TRUE/FALSE/UNKNOWN)로 올바르게 비교한다 |
| 목표 ⑤ | ORDER BY·LIMIT과 내장 함수로 결과를 정렬·가공한다         |

지난주 적재한 다운유통 시드 데이터로 조회를 시작합니다

# 오늘 다루는 일곱 가지

---

1 SELECT 문의 기본 구조

2 컬럼 선택과 별칭

3 WHERE 절 — 조건 검색

4 NULL의 처리

5 ORDER BY — 정렬

6 LIMIT — 결과 제한과 페이징

7 자주 쓰는 내장 함수

## 1. SELECT 문의 기본 구조

# SELECT 문 전체 구문 — 쓰는 순서

|   |          |                 |               |
|---|----------|-----------------|---------------|
| 1 | SELECT   | [DISTINCT] 컬럼목록 | -- ⑤ 표시할 열 선택 |
| 2 | FROM     | 테이블명            | -- ① 대상 테이블   |
| 3 | WHERE    | 조건              | -- ② 행 필터링    |
| 4 | GROUP BY | 컬럼              | -- ③ 그룹화      |
| 5 | HAVING   | 그룹조건            | -- ④ 그룹 필터링   |
| 6 | ORDER BY | 컬럼 [ASC DESC]   | -- ⑥ 정렬       |
| 7 | LIMIT    | 개수 [OFFSET 시작]; | -- ⑦ 결과 제한    |

동그라미 숫자 = 실제 실행 순서 · 대괄호는 생략 가능 · GROUP BY·HAVING은 6주차에 다룬다

1. SELECT 문의 기본 구조

# 작성 순서와 실행 순서는 다르다

| 실행 순서 | 절           | 역할                  |
|-------|-------------|---------------------|
| 1     | FROM / JOIN | 데이터를 가져올 테이블 결정     |
| 2     | WHERE       | 행 단위 필터링 (별칭 사용 불가) |
| 3     | GROUP BY    | 지정한 컬럼으로 그룹화        |
| 4     | HAVING      | 그룹 단위 필터링           |
| 5     | SELECT      | 표시할 컬럼 선택, 별칭 부여    |
| 6     | ORDER BY    | 정렬 (별칭 사용 가능)       |
| 7     | LIMIT       | 결과 행 수 제한           |

FROM이 가장 먼저, SELECT는 거의 마지막 — 그래서 SELECT의 별칭은 WHERE에서는 못 쓰고 ORDER BY에서는 쓴다

## 2. 컬럼 선택과 별칭

# 어느 테이블에서 어떤 열을 보여줄까

```
1  -- 전체 컬럼
2  SELECT * FROM customers;
3
4  -- 특정 컬럼
5  SELECT cust_id, name, points FROM customers;
6
7  -- 별칭(AS) — AS는 생략 가능
8  SELECT cust_id AS 고객번호,
9         name    AS 이름,
10        points  AS 포인트
11 FROM customers;
```

1

2행

1

**\* = 모든 컬럼**

구조를 훑어볼 때만 — 보고서에는 필요한 열만

2

5행

2

**세 열만 고르기**

열 순서는 적은 순서대로 나온다

3

8-11행

3

**열 제목을 한글로**

결과 화면만 바뀌고 원본 컬럼명은 그대로

## 2. 컬럼 선택과 별칭

# 계산식도 열이 된다 — 산술과 문자열 연결

```
1  -- 공백이 포함된 별칭은 따옴표로 감싼다
2  SELECT name AS '고객 이름', price * 0.9 AS '10% 할인가'
3  FROM products;
4
5  -- 산술 연산과 문자열 연결
6  SELECT name,
7         price * stock AS 재고자산액,
8         CONCAT(name, '(', prod_id, ')') AS 표기
9  FROM products;
```

2행

- 1 10% 할인가  
price \* 0.9를 계산해 보고서 열로

7행

- 2 단가 × 재고  
창고에 묶인 금액

8행

- 3 CONCAT()  
무선 전기포트(P001)처럼 이어 붙인다

MySQL 문자열 연결은 || 가 아니라 CONCAT() — Oracle·PostgreSQL은 || 를 쓴다

## 2.1 DISTINCT — 중복 제거

# 같은 값이 반복될 때 종류만 보기

```
1  -- 고객 등급 목록 (중복 포함, 10행)
2  SELECT grade FROM customers;
3
4  -- 중복 제거 → VIP, GOLD, SILVER (3행)
5  SELECT DISTINCT grade FROM customers;
6
7  -- 여러 컬럼의 조합에 대해 중복 제거
8  SELECT DISTINCT grade, city FROM customers;
9
10 -- 중복 제거 후 개수 세기 (NULL은 제외됨)
11 SELECT COUNT(DISTINCT city) AS 도시수 FROM customers;
12 -- 서울 · 부산 · 대전 · 광주 · 인천 → 5 (C007의 city NULL은 제외)
```

5행

### 1 등급 체계 확인

어떤 등급이 있나에 답할 때

8행

### 2 (등급, 도시) 짝

둘 다 같아야 한 행으로 합쳐진다

11-12행

### 3 COUNT(DISTINCT)

서로 다른 값의 개수 — 고객이 분포한 도시 수

# 비교 연산자 — 경영 질문을 조건 하나로

| 연산자                   | 의미                        | 예시                                 |
|-----------------------|---------------------------|------------------------------------|
| =                     | 같다                        | WHERE grade = 'VIP'                |
| <>, !=                | 같지 않다                     | WHERE status <> '취소'               |
| >, >=, <, <=          | 크다 / 크거나 같다 / 작다 / 작거나 같다 | WHERE price >= 20000               |
| BETWEEN a AND b       | a 이상 b 이하 (경계 포함)         | WHERE points BETWEEN 1000 AND 3000 |
| IN (값목록)              | 목록 중 하나와 일치               | WHERE category IN ('가전','식품')      |
| LIKE 패턴               | 패턴 일치 (문자열)               | WHERE name LIKE '김%'               |
| IS NULL / IS NOT NULL | NULL 여부                   | WHERE emp_id IS NULL               |

「서울의 VIP에게 쿠폰을」 「품질 상품을 발주하자」 — 타겟 마케팅과 재고 관리의 출발점

### 3.2 논리 연산자

# AND · OR · NOT — 조건 이어 붙이기

```
1  -- AND: 두 조건 모두 만족 — 서울에 사는 VIP 고객
2  SELECT name, city, points
3  FROM    customers
4  WHERE   grade = 'VIP' AND city = '서울';
```

1

4행

1 **AND**

두 조건을 다 만족하는 행만 — 타겟 마케팅

```
5  -- 결과: 한가람
```

```
6
7  -- OR: 둘 중 하나 만족
8  SELECT name FROM customers
9  WHERE   grade = 'VIP' OR grade = 'GOLD';
```

2

9행

2 **OR**

VIP이거나 GOLD — 우수 고객군 전체

```
10
11 -- NOT: 조건 부정
12 SELECT name FROM customers WHERE NOT grade = 'SILVER';
13 SELECT name FROM products  WHERE category NOT IN ('가전', '식품');
```

3

12-13행

3 **NOT**

SILVER가 아닌 고객 · 가전·식품을 뺀 상품

### 3.2 논리 연산자

## 우선순위 NOT > AND > OR — OR는 괄호로

```
1  -- 우선순위 주의: 아래 두 결과는 다르다
2  SELECT * FROM customers WHERE grade='VIP' OR grade='GOLD' AND city='서울';
3  --   → grade='VIP' OR (grade='GOLD' AND city='서울')
4
5  SELECT * FROM customers WHERE (grade='VIP' OR grade='GOLD') AND city='서울';
6  --   → 서울에 사는 VIP · GOLD 고객
```

1

2-3행

1

**괄호 없음**

AND가 먼저 묶여 서울이 아닌 VIP(임수빈·대전)도 들어온다

2

5-6행

2

**OR를 괄호로**

의도가 「서울에 사는 우수 고객」이면 이쪽

연산자 우선순위는 NOT > AND > OR — OR 조건은 괄호로 묶어 의도를 분명히 한다

### 3.3 BETWEEN과 IN

## 범위는 BETWEEN, 목록은 IN

```
1  -- BETWEEN은 양쪽 경계를 포함한다 (>= AND <=)
2  SELECT name, price FROM products WHERE price BETWEEN 10000 AND 20000;
3  -- 결과: 유기농 원두 500g, 곡물 시리얼 세트, 천연 세탁세제
4
5  -- 날짜에도 사용 가능 — 2024년에 가입한 고객
6  SELECT name, joined_date FROM customers
7  WHERE joined_date BETWEEN '2024-01-01' AND '2024-12-31';
8  -- 결과: 한가람, 오세영, 장민혁
9
10 -- NOT BETWEEN
11 SELECT name FROM products WHERE price NOT BETWEEN 5000 AND 50000;
12
13 -- IN: 여러 값 중 하나
14 SELECT name, category FROM products WHERE category IN ('가전', '문구');
```

1

2행

1

**1만~2만 원대**

10000원·20000원짜리도 포함된다

2

7행

2

**날짜 범위**

'YYYY-MM-DD' 문자열 — 12-31까지 한 해 전체

3

11행

3

**범위 바깥**

5000·50000 자체는 범위 안이라 제외

4

14행

4

**IN**

= category='가전' OR category='문구'

# 와일드카드 % 와 \_

| 와일드카드  | 의미            | 예시                  | 매칭                  |
|--------|---------------|---------------------|---------------------|
| %      | 0개 이상의 임의 문자  | '김%'                | 김도현, 김, 김성호         |
| _      | 정확히 1개의 임의 문자 | '김_'                | 김철 (2글자만)           |
| %문자%   | 포함            | '%세트%'              | 곡물 시리얼 세트, 3색 볼펜 세트 |
| ESCAPE | 와일드카드 자체 검색   | '100\%' ESCAPE '\\' | 100%                |

= 는 완전히 같은 문자열만 찾는다 — 김씨 고객, 세트 상품처럼 부분 일치가 필요하다면 LIKE

### 3.4 LIKE — 패턴 검색

## 문자열 일부로 찾기

```
1 SELECT name FROM customers WHERE name LIKE '김%';
2 -- 결과: 김도현
3
4 SELECT name FROM customers WHERE name LIKE '%연';
5 -- 결과: 박서연
6
7 SELECT name, email FROM customers WHERE email LIKE '%@example.com';
8
9 SELECT name FROM employees WHERE name LIKE '___';
10
11 SELECT name FROM products WHERE name LIKE '%세트%';
12 -- 결과: 곡물 시리얼 세트, 3색 볼펜 세트
```

1

1-5행

- 1 시작 '김%' · 끝 '%연'  
%는 글자 수 상관없이(0개 포함)

2

7행

- 2 이메일 도메인  
email NULL인 C005는 UNKNOWN이라 빠진다

3

9행

- 3 밑줄 세 개  
정확히 3글자 이름인 직원

4

11행

- 4 양쪽 %  
위치와 상관없이 포함

패턴이 % 로 시작하면 인덱스를 쓸 수 없어 전체 테이블을 검색한다 (10주차 인덱스)

#### 4. NULL의 처리

# NULL은 0도 빈 문자열도 아니다

## 값 없음·알 수 없음

01

숫자 0이나 빈 문자열과  
전혀 다른 특수한 표시

## 다온유통의 NULL

02

온라인 주문 emp\_id  
협상 중 salary · 미입력 city

## 비교 결과는 UNKNOWN

03

NULL과의 모든 비교는  
참도 거짓도 아니다

WHERE는 참인 행만 남긴다 — UNKNOWN인 행은 조용히 빠진다

#### 4. NULL의 처리

# = NULL 이 아니라 IS NULL

```
1  -- 잘못된 방법 — 결과가 항상 비어 있다
2  SELECT order_id FROM orders WHERE emp_id = NULL;    -- (X)
3
4  -- 올바른 방법
5  SELECT order_id FROM orders WHERE emp_id IS NULL;    -- 온라인 주문 (7건)
6  SELECT order_id FROM orders WHERE emp_id IS NOT NULL; -- 오프라인 주문
```

2행

- 1 **= NULL은 0행**  
NULL = NULL 조차 UNKNOWN — 오류도 나지 않는다

5행

- 2 **IS NULL**  
담당 직원 없는 온라인 주문 1002·1004·...·1014

6행

- 3 **IS NOT NULL**  
영업 담당 주문 — 직원별 실적 집계 대상

0행이 나오고 오류도 나지 않는다 — 가장 위험한 실수

#### 4. NULL의 처리

# NULL 찾기와 NULL 대체하기

```
1  -- 연봉이 아직 정해지지 않은 직원
2  SELECT name, dept FROM employees WHERE salary IS NULL;
3  -- 결과: 조하늘
4
5  -- NULL과의 산술 연산 결과는 NULL
6  SELECT name, salary, salary + 50 AS 인상후 FROM employees;
7
8  -- NULL을 대체하여 조회
9  SELECT name, IFNULL(salary, 0) AS 연봉 FROM employees;
10 SELECT name, COALESCE(salary, 0) AS 연봉 FROM employees; -- 표준 SQL
```

1

2행

1 시드의 E008

salary NULL은 조하늘만 해당

2

6행

2 알 수 없는 값 + 50

여전히 NULL — 계산에 섞이면 결과가 사라진다

3

9행

3 IFNULL(값, 대체값)

보고서에 빈칸 대신 0을 찍을 때

4

10행

4 COALESCE

NULL이 아닌 첫 값 — 다른 DBMS에서도 통한다

4. NULL의 처리

# 3값 논리 — TRUE · FALSE · UNKNOWN

| 조건                 | salary = 380 (이지은) | salary = NULL (조하늘) |
|--------------------|--------------------|---------------------|
| salary > 350       | TRUE → 조회됨         | UNKNOWN → 조회 안 됨    |
| salary <= 350      | FALSE → 조회 안 됨     | UNKNOWN → 조회 안 됨    |
| salary IS NULL     | FALSE              | TRUE → 조회됨          |
| NOT (salary > 350) | FALSE              | UNKNOWN → 조회 안 됨    |

salary > 350 과 salary <= 350 어디에도 NULL 행은 없다 — 빠짐없이 다루려면 OR salary IS NULL 을 명시

## 5. ORDER BY — 정렬

# 정렬 기본 — 오름·내림·다중 정렬

```
1  -- 오름차순 (ASC는 기본값이므로 생략 가능)
2  SELECT name, price FROM products ORDER BY price ASC;
3
4  -- 내림차순 — 포인트가 많은 고객부터
5  SELECT name, points FROM customers ORDER BY points DESC;
6
7  -- 다중 정렬: 등급 오름차순 → 같은 등급 안에서 포인트 내림차순
8  SELECT name, grade, points
9  FROM customers
10 ORDER BY grade ASC, points DESC;
```

2행

- 1 **싼 상품부터**  
가격표 순서

5행

- 2 **DESC**  
큰 값이 먼저 — 우수 고객 순위표

10행

- 3 **두 번째 기준**  
첫 기준이 같은 행끼리만 다시 정렬

ORDER BY가 없으면 행 순서는 보장되지 않는다

## 5. ORDER BY — 정렬

# 별칭으로 정렬, 순번으로 정렬

```
1 -- 별칭으로 정렬 (ORDER BY는 SELECT 이후 실행되므로 가능)
2 SELECT name, price * 0.9 AS 할인가
3 FROM products
4 ORDER BY 할인가 DESC;
5
6 -- 컬럼 순번으로 정렬 (2번째 컬럼 기준) — 가독성이 떨어져 권장하지 않음
7 SELECT name, price FROM products ORDER BY 2 DESC;
```

4행

### 1 별칭 사용 가능

WHERE에서는 아직 만들어지기 전이라 불가

7행

### 2 2 = 두 번째 열

SELECT 목록을 고치면 정렬 기준이 조용히 바뀐다

## 5.1 NULL의 정렬 위치

# 연봉 미정을 맨 뒤로 보내는 요령

```
1  -- MySQL은 ASC에서 NULL이 가장 앞, DESC에서 가장 뒤에 온다
2  SELECT name, salary FROM employees ORDER BY salary;
3  -- 조하늘(NULL)이 첫 행
4
5  -- IS NULL 결과(0/1)로 먼저 정렬하여 NULL을 뒤로 보낸다
6  SELECT name, salary
7  FROM   employees
8  ORDER BY salary IS NULL, salary DESC;
9
10 -- 표준 SQL (Oracle, PostgreSQL)은 NULLS LAST를 지원한다
11 -- SELECT name, salary FROM employees ORDER BY salary DESC NULLS LAST;
```

2행

- 1 NULL = 가장 작은 값처럼  
오름차순이면 첫 줄에 나온다

8행

- 2 1차 NULL 여부, 2차 연봉  
값 있음(0)이 먼저, NULL(1)이 뒤로

11행

- 3 MySQL에는 없다  
그래서 IS NULL을 정렬 기준으로 대신 쓴다

## 6. LIMIT — 결과 제한과 페이징

# 상위 N명과 페이지 나누기

```
1  -- 포인트 상위 3명
2  SELECT name, points FROM customers ORDER BY points DESC LIMIT 3;
3  -- 결과: 임수빈(5200), 한가람(4500), 오세영(2300)
4
5  -- 4~6등 (앞의 3건을 건너뛰고 3건)
6  SELECT name, points FROM customers ORDER BY points DESC LIMIT 3 OFFSET 3;
7
8  -- MySQL 축약형: LIMIT 시작위치, 개수
9  SELECT name, points FROM customers ORDER BY points DESC LIMIT 3, 3;
```

2행

### 1 정렬한 뒤 자르기

줄 세운 뒤 위에서 3행만 남긴다

6행

### 2 OFFSET 3

앞 3행을 건너뛰고 4등부터 3행

9행

### 3 건너뛸 수, 가져올 수

OFFSET 형과 반대로 읽힌다 — 헛갈리면 OFFSET

# LIMIT을 쓸 때 기억할 세 가지

## ORDER BY와 함께

01

정렬 없이 LIMIT만 쓰면  
반환되는 행이 매번 달라질 수 있다

## 페이징 공식

02

LIMIT size OFFSET (N-1) \* size  
2페이지·5건 → LIMIT 5 OFFSET 5

## MySQL 밖에서는

03

LIMIT 대신 Oracle FETCH FIRST  
SQL Server TOP

1페이지는 OFFSET 0, 3페이지는 OFFSET 10 — 시작 위치가 0부터 세어진다

# 붙이고, 자르고, 찾고, 바꾸기

| 함수                         | 기능              | 예시                           | 결과       |
|----------------------------|-----------------|------------------------------|----------|
| CONCAT(s1,s2,...)          | 문자열 연결          | CONCAT('다운','유통')            | 다운유통     |
| CONCAT_WS(구분자,...)         | 구분자로 연결         | CONCAT_WS('-', '010','1234') | 010-1234 |
| LENGTH(s) / CHAR_LENGTH(s) | 바이트 수 / 글자 수    | CHAR_LENGTH('다운유통')          | 4        |
| UPPER(s) / LOWER(s)        | 대·소문자 변환        | UPPER('sql')                 | SQL      |
| SUBSTRING(s,시작,길이)         | 부분 문자열 (1부터 시작) | SUBSTRING('DATABASE',1,4)    | DATA     |
| LEFT(s,n) / RIGHT(s,n)     | 왼쪽·오른쪽 n글자      | LEFT('C001',1)               | C        |
| TRIM(s) / LTRIM / RTRIM    | 공백 제거           | TRIM(' db ')                 | db       |
| REPLACE(s,old,new)         | 문자열 치환          | REPLACE('A-B','-','/')       | A/B      |
| LPAD(s,n,p) / RPAD         | 지정 길이로 채우기      | LPAD('7',3,'0')              | 007      |
| INSTR(s,부분)                | 위치 반환 (없으면 0)   | INSTR('database','base')     | 5        |

# 저장된 문자열을 보고서 형태로

```
1  -- 고객 이름과 등급을 결합하여 표시
2  SELECT CONCAT(name, ' (', grade, ')') AS 고객표기 FROM customers;
3
4  -- 이메일에서 아이디 부분만 추출
5  SELECT email,
6         SUBSTRING(email, 1, INSTR(email, '@') - 1) AS 아이디
7  FROM    customers
8  WHERE   email IS NOT NULL;
9
10 -- 성(첫 글자)만 추출
11 SELECT LEFT(name, 1) AS 성, name FROM customers;
```

2행

- 1 네 조각 잇기  
예: 한가람 (VIP)

6행

- 2 @ 앞까지 자르기  
INSTR로 @ 위치를 찾고 그 앞 글자까지

8행

- 3 NULL 미리 빼기  
C005는 email NULL이라 INSTR도 NULL

11행

- 4 LEFT(name, 1)  
왼쪽 한 글자 — 한국 이름의 성씨

# 반올림·버림·올림·나머지

| 함수                 | 기능         | 예시                    | 결과      |
|--------------------|------------|-----------------------|---------|
| ROUND(n,자릿수)       | 반올림        | ROUND(3.856, 1)       | 3.9     |
| TRUNCATE(n,자릿수)    | 버림         | TRUNCATE(3.856, 1)    | 3.8     |
| CEIL(n) / FLOOR(n) | 올림 / 내림    | CEIL(3.1), FLOOR(3.9) | 4, 3    |
| ABS(n)             | 절댓값        | ABS(-7)               | 7       |
| MOD(a,b) 또는 a % b  | 나머지        | MOD(10,3)             | 1       |
| POW(a,b) / SQRT(n) | 거듭제곱 / 제곱근 | POW(2,10), SQRT(16)   | 1024, 4 |

# 날짜 추출·변환·계산

| 함수                         | 기능          | 예시                                      |
|----------------------------|-------------|-----------------------------------------|
| NOW() / SYSDATE()          | 현재 날짜와 시각   | 2026-08-30 14:30:00                     |
| CURDATE() / CURTIME()      | 현재 날짜 / 시각  | 2026-08-30 / 14:30:00                   |
| YEAR(d), MONTH(d), DAY(d)  | 연·월·일 추출    | YEAR('2026-03-02') → 2026               |
| DATE_FORMAT(d, 포맷)         | 날짜 형식 변환    | DATE_FORMAT(NOW(), '%Y년 %m월 %d일')       |
| DATEDIFF(d1,d2)            | 두 날짜의 일수 차이 | DATEDIFF('2026-03-02','2026-03-01') → 1 |
| DATE_ADD(d, INTERVAL n 단위) | 날짜 더하기      | DATE_ADD(CURDATE(), INTERVAL 7 DAY)     |
| TIMESTAMPDIFF(단위,d1,d2)    | 단위별 차이      | TIMESTAMPDIFF(YEAR, joined_date, NOW()) |

### 7.3 날짜 함수

## 가입일 다듬기와 기간 계산

```
1 SELECT name,  
2     DATE_FORMAT(joined_date, '%Y년 %m월 %d일') AS 가입일,  
3     TIMESTAMPDIFF(YEAR, joined_date, CURDATE()) AS 가입연차  
4 FROM customers  
5 ORDER BY joined_date;  
6  
7 -- 2024년에 가입한 고객  
8 SELECT name FROM customers WHERE YEAR(joined_date) = 2024;  
9 -- 결과: 한가람, 오세영, 장민혁  
10  
11 -- 주문일을 월 단위로 표기 (6주차 월별 집계 자료)  
12 SELECT order_id, DATE_FORMAT(order_date, '%Y-%m') AS 주문월 FROM orders;
```

1

2-3행

1 형식 변환과 가입 연차

2024년 03월 12일 형태 · 오늘까지 만 몇 년

2

8행

2 YEAR()로 연도만

BETWEEN '2024-01-01' AND '2024-12-31'과  
같은 결과

3

12행

3 '%Y-%m'

같은 달 주문이 같은 값 — GROUP BY의 기준

컬럼에 함수를 씌우면 인덱스를 못 쓴다 — 성능이 중요하다면 `joined_date >= '2024-01-01' AND joined_date < '2025-01-01'`

# 확인 퀴즈

---

SQL 문의 실행 순서로 옳은 것은?

① SELECT → FROM → WHERE → ORDER BY

② FROM → WHERE → SELECT → ORDER BY

③ WHERE → FROM → SELECT → ORDER BY

④ FROM → SELECT → WHERE → ORDER BY

# 정답 및 해설

---

정답

② FROM → WHERE → SELECT → ORDER BY

해설

실행 순서는 FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT 입니다.

# 핵심 요약

|      |                                                                    |
|------|--------------------------------------------------------------------|
| 핵심 ① | 실행 순서 FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT |
| 핵심 ② | SELECT의 별칭은 WHERE에서 불가, ORDER BY에서 가능                              |
| 핵심 ③ | BETWEEN은 양쪽 경계 포함, LIKE의 %는 0개 이상, _는 정확히 1개                       |
| 핵심 ④ | NULL 비교는 IS NULL / IS NOT NULL — = NULL은 0행                        |
| 핵심 ⑤ | LIMIT은 항상 ORDER BY와 함께, 문자열 연결은 CONCAT()                           |

**NULL과의 비교는 UNKNOWN — 오류 없이 행이 빠진다는 점을 기억해 두세요**

다음 주차 예고

# 6주차 · 집계와 그룹화

01

## 집계 함수

COUNT·SUM·AVG  
NULL을 세는 법

02

## GROUP BY

등급별·월별  
그룹으로 묶기

03

## HAVING

그룹에 거는  
조건

04

## 소계와 총계

WITH ROLLUP  
취소 주문 제외 매출

5주차 마무리

## 학습 마무리

조회의 기본기를 다음 주 집계·그룹화로 이어 갑니다

✓ 작성 순서와 실행 순서

✓ IS NULL과 3값 논리

✓ WHERE 조건과 LIKE 패턴

✓ ORDER BY·LIMIT·내장 함수

이어나는 활동

5주차 실습 10문항 · 퀴즈 4